Spring 2026	C435 Operating Systems ULTIMA 2.0 Phase III – Memory Management Unit (MMU)	H. Hakimzadeh Consult Canvas for Due Date

During the last phase two phases you implemented the Scheduler for process management, a Semaphore for managing critical section code and synchronization, followed by Message Passing for inter-process communication (IPC). During this phase, your team will design and implement memory management for ULTIMA 2.0. Memory management allows tasks to dynamically ask for or free memory.

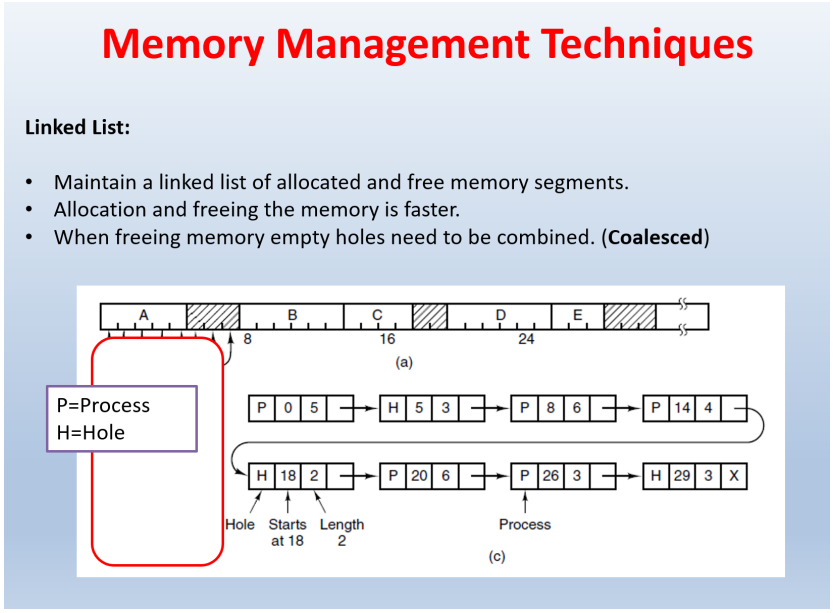
PHASE 3: Memory Management Unit

Assuming ULTIMA 2.0 has access to only **1024 bytes of RAM** available to it, develop a memory management (MMU) class that allows task to ask for memory, write to it, read from it, and free it when they are done. The Memory Management Unit (MMU) may be implemented as an independent thread (serving memory to its clients) or it may be implemented as part of the main thread ("the OS"). Your MMU must provide two public functions for allocation and deallocation of memory. Mem_Alloc() and Mem_Free(). Several other functions will also be needed for internal debugging, dump, and clean-up operations. When implementing Mem_Read() and Mem_Write(), you may want to think ahead about possible virtual memory ideas and consider the use of BASE and LIMIT registers to allow for relocation and protection.

```
class mmu {
    mmu(int size, char default_initial_value, int page_size); // allocate 1024 unsigned chars and initialize the entire memory with "." dots.
    ~mmu();                                                   // Free up the memory and its related structures

    int Mem_Alloc(int size);                                  // returns a unique integer mem_handle or -1 if not enough memory is
                                                              available. Consider fixed size BLOCK allocation (e.g., blocks of size
                                                              64 or 128 bytes). So, if the user asks for 35 byte, you allocate 1 block,
                                                              if they ask for blocksize+1, you allocate two blocks for them. NOTE:
                                                              The mem_handle should be kept somewhere in the requesting task's
                                                              TCB.

    int Mem_Free(int memory_handle);                          // place "#" in the memory the is being freed. Return -1 if error
                                                              occurs. NOTE: that initially, the memory is full of "dots", but as
                                                              memory is being freed, we place "#" in free location. This should
                                                              allow you to more easily debug your program.

    int Mem_Read(int memory_handle, char *ch);                // read a character from current location in memory and bring it back
                                                              in "ch", return a -1 if at end of bounds. NOTE: you need to keep
                                                              track of the "Current Location" or the location of the next char to be
                                                              read. (where is a good place for this?)

    int Mem_Write(int memory_handle, char ch);                // write a character to the current_location in memory, return a -1 if
                                                              at end of bounds.

    //Overloaded multi-byte read and write.
    int Mem_Read(int memory_handle, int offset_from_beg, int text_size, char *text )
    int Mem_Write((int memory_handle, int offset_from_beg, int text_size, char *text )

    Your Memory Manager must also include the following private functions for internal debugging, status, and clean-up operations.

    int Mem_Left();                                           // return the amount of core memory left in the OS
    int Mem_Largest();                                        // return the size of largest available memory segment.
    int Mem_Smallest();                                       // return the size of smallest available memory segment.
    int Mem_Coalesce();                                       // combine two or more contiguous blocks of free space, and place
                                                              "." Dots in the coalesced memory.
    int Mem_Dump(int starting_from, int num_bytes);           // dump the contents of memory (typically we dump the entire
                                                              memory)
}
```

Other Implementation Issues:

- When task ask for and later free up memory, you must create and maintain a corresponding **linked list** of free and used blocks of memory. See Unit 3a regarding memory management techniques. As for memory management algorithm, use the one you wish (I recommend **first-fit**)



- **Access to memory should be protected.** Memory is a shared resource and therefore you should be careful when you try to allocate it. However, once you allocate it, it belongs to you and you can write to your memory concurrently with other tasks. (since you are not interfering with each other). Of course, if you try to access someone else's memory space (accidentally or maliciously), that should not be allowed, and you should generate a segmentation fault. In other words, if your handle/task-id combination is not valid, or if you try to read or write to memory before you BASE or beyond your LIMIT, you should get an error.
- **When a task wants to mem_read()** one or more characters we must remember the following:
    - Single read of a character should occur from the "Current_Location" of that memory handle and bring it back in "ch", return a -1 if at end of bounds. Once the character is read, the "Current_Location" must be adjusted.
    - Multi-character read should occur from (starting_location+offset_from_beg) and continue for (text_size). The result should be returned in (char array) container.
    - Review the concept of base and limit registers. (relocation and protection) Although we are using that concept in slightly different context!
- **When a task wants to mem_write(),** similar issues as mem_read() must be observed.
- **SEMAPHORE:** In order for the Memory Management to work properly, you must implement a **MEMORY SEMAPHORE.** Otherwise, corruption of the core memory or the linked list structures that hold information about which task has which memory blocks can occur. (i.e., two tasks may end up pointing to the same memory space.)
- **When freeing the memory** consider having the Mem_Free() function Un-Block all other processes waiting for memory, allowing them to once again compete for the available memory. This method is often referred to as "Shark Pond" memory allocation technique. (Watch out for the possibility of race conditions)
- **Consider Allocating memory in 64 or 128-byte blocks.** (it will make life easier, and it will also make your mem_dump() function easier to understand.
- **Use first fit** memory allocation strategy.
- **The Mem_Coalesce() should be called internally**, every time the Mem_Free() function is called by the user. However, you should be able to pause the OS, dump your memory before and after mem_coalesce(). This allows you to see a deleted segment represented as "#########", followed by coalesced segment represented as "……"

- If you are using the concept of MCB, then you may want to add your Memory manager to the structure

```
struct MCB {                          // Master Control Block
    scheduler     Swapper;
    ipc           Messenger;
    mmu           MemMgr(1024, '.');
    semaphore     Monitor("Monitor", 1);
    semaphore     Printer("Printer", 1);
    semaphore     Core("Core Memory", 1);

}
```

- You may have to add additional functionality to your scheduler to accommodate the Memory Manager. (for example, the notion of "current location", "memory_handle", etc may have to be preserved in the task's TCB. Brainstorm with your team!
- Like the previous phases, this phase requires some brainstorming by your team. So, do not wait. Start today. Also, please feel free to contact me for discussion and brainstorming sessions.

Note:

- Develop an estimate for how much time this phase requires. Use an approach like the one we discussed in class. Think about all the various components, classes, proof of concepts, external research, group meetings, etc., and associate an amount of time to each. Perhaps a minimum and maximum amount. (that should give a fair average estimate). Then depending on your own assessment of your experience, multiply your estimate by a factor (a factor of 2 or 3 is not uncommon). Now you have a little better estimate of the time it will take to do the phase. As you move forward your ability to estimate will be refined.

Expected Output:

- To show that your memory manager is working properly, you must clearly show that several tasks ask for and receive memory.
- You must also show what will happen if a task asks for but cannot receive memory.
- You will need to show the linked list of nodes which maintain the free and used spaces, as well as the core itself.

Possible output of the Memory Usage (info in the linked list):

Status	Memory Handle	Starting Location	Ending Location	Size/ Bytes	Current Location	Task-ID
Used	1	0	127	128	17	1
Free	2	128	1023	896	NA	MMU

MMU = Memory Manager

Memory CORE Dump:

```
this is task one.....
.....
.....
.....
.....
.....
.....
.....
```

- Spend some time to think about the best approach for testing this phase. Note that you want your test to be cumulative. In other words, you still want to occasionally dump your scheduler to show which tasks are running, ready, blocked, etc., who has access to what semaphores, who is in which semaphore queues, who has sent messages to who, and of course who allocated what memory and has read/written what.

Managing Your Workflow:

- Each group should create one main folder called ULTIMA, with the following sub folders. Please place this folder in a cloud space that is shared by your team members (and possibly me if I ask for it) so that everyone in your team has access to all the resources.

  ULTIMA

  Introduction
  - Cover page with your group name and number, course # and name, project name, date, group names, e-mails, etc.
  - Project Abstract (Summary of accumulated phase abstracts. With each phase this will be supplemented)
  - Design diagram (Summary of accumulated phase design diagrams. With each phase this will be supplemented)
  - Resume from each team member (use the one from your profile submitted in discussion)

  Phase 1 (Scheduler)
  Phase 2 (Message Passing)
  Phase 3 (Memory Management)
  - Copy of the assignment sheet
  - Phase Abstract
  - External Documentation (for this phase):
    - Design diagram for this phase (including the data structures and class hierarchy, nicely drawn using a drawing package)
  - Source files:
    - With appropriate documentation, each class/module should have a primary author, as well co-authors, if appropriate.
  - Program execution/testing:
    - A Word document showing the submission information including screenshots, code snippets, test cases, etc.
  - Each phase should be completely self-contained (all the files needed to compile and run the phase) (note that after phase 1, all phases are cumulative, in other words, phase 2 depends on phase 1, and it must include phase I.
  - Video with narration for the phase.

  Phase 4 (File Systems)
  Extra Credit Phase
  Option 1: (Create and integrate a window and window management for ULTIMA)
  Option 2: (Implement Security/Privacy/Encryption for one or more of the previous phases.)
  (e.g., Secure IPC, Secure Memory Management, Secure File System)
  (restricted access, and encrypted data)

What to hand in Canvas:

- A zipped copy of the appropriate folders for the phase:
  - Introduction folder
  - Phase 1 folder (Scheduler/Semaphore)
  - Phase 2 folder (IPC)
  - Phase 3 folder (Memory Management)

- A digital version of the completed Self and Peer evaluation form. (DO NOT place this form in the public area, it is not meant to be shared with your team members. This is your communication to me)

Self / Peer Evaluation Form
C435 - Operating Systems
ULTIMA 2.0

Your Name		Team Number:		Team Name:	
Date:		Phase Number:		Phase Title:	

What were your **exact responsibilities** for this phase?
A) Design Activity: (Name the functions, data structures, classes, algorithms you designed or help design)
B) Implementation Activity: (What functions, data structures, classes, algorithms you implemented or help implement)
C) Documentation Activity: (What functions, data structures, classes, or overall project did you document? Documentation could be classified as **internal** to a function, or to a class, or **external** about the phase or the overall project.)
D) Testing Activity: (What functions, data structures, classes, or overall project did you test?) Testing could be classified as unit testing (e.g., function or class) or overall/integration testing (phase level, or project level)
E) Other Activities: (not mentioned above, that you would like to highlight)

How would you improve the system you have implemented? (Be specific)

Do you understand the design decisions that were made by your group? (explain if you agree or disagree with the design decisions)

What did you learn during this phase of the project?

Which of the following helped you to understand and complete this phase of the project? (e.g., Textbook, Class videos/PowerPoint, labs, class discussion, group discussions, external sources). If external sources, please specify.

Given the following categories, evaluate yourself and your team member. Estimate the contributions based both in terms of hours and percentages of total work. *(For example: I attended all the meeting 100%, 4 hours, I implemented a small class, but the rest of the classes were implemented by my team members, so I estimate my contribution as 25% and 3 hours, I did most of the documentation, so my contribution was 90% and spent 3 hour)*

YOU	Estimated # of Hours Spent (during the phase)	Estimated % of your contribution to the overall work
a) Attending group meetings		
b) Participation in the design		
c) Participation in the implementation		
d) Participation in the debugging		
e) Participation in testing		
f) Participation in the documentation		
g) Overall contribution to the group		
h) Other Activities (Explain below):		

Your Team Member's Name:	Estimated # of Hours Spent (during the phase)	Estimated % of her/his contribution to the overall work
a) Attending group meetings		
b) Participation in the design		
c) Participation in the implementation		
d) Participation in the debugging		
e) Participation in testing		
f) Participation in the documentation		
g) Overall contribution to the group		
h) Other Activities (Explain below):		

Your Team Member's Name:	Estimated # of Hours Spent (during the phase)	Estimated % of her/his contribution to the overall work
a) Attending group meetings		
b) Participation in the design		
c) Participation in the implementation		
d) Participation in the debugging		
e) Participation in testing		
f) Participation in the documentation		
g) Overall contribution to the group		
h) Other Activities (Explain below):		